Automata Theory

Pushdown Automata

CS32022

By Thisara Watawana

16 September 2026

5. Pushdown Automata

Exercise 01:

- Design a Deterministic Finite Automaton (DFA) over the alphabet Σ = {0, 1} that accepts the following language:

  $$B = \{ 0^n 1^n \mid n >= 0 \} = \{ \varepsilon, 01, 0011, 000111, 00001111, ... \}$$

- Draw a state transition diagram for a machine that accepts strings where every block of 0s is followed by an equal number of 1s.
- Ensure your machine works for any arbitrary value of n (e.g., n = 1, n = 5, n = 100).

5. Pushdown Automata

The Limits of Finite State:

- Recognizing strings with matching counts of 0s and 1s requires tracking an arbitrarily large number of inputs.
- Fixed Finite Capacity: A Deterministic Finite Automaton possesses a fixed, finite number of states (p) by definition.
- Processing any string containing p or more 0s forces the machine to revisit at least one state it has already seen.
- Revisiting a state creates a loop in the machine that cannot track how many times it was traversed.
- Inevitable Memory Breakdown: A finite state machine cannot distinguish between different counts of zeros once it starts looping.

5. Pushdown Automata

The Pigeonhole Principle in Automata:

- If you place n items into m containers and n > m, at least one container must hold more than one item.
- Processing a string of length p requires a sequence of p + 1 total state visits.
- Traversing p + 1 state visits using only p available states guarantees at least one state is visited twice.
- **Creation of Loops**: The sequence of transitions between the first and second visit to that repeated state forms an inescapable loop.

5. Pushdown Automata

The Pumping Lemma Theorem

- Every regular language possesses a critical length p where any string of length at least p can be pumped.
- If a language L is regular, there exists a constant integer $p \geq 1$. For any string $w \in L$ whose length is at least p ($|w| \geq p$), we can split w into three contiguous pieces $w = xyz$ such that:
  - y can be repeated ("pumped") indefinitely: $xy^iz \in L$ for every integer $i \geq 0$.
  - The loop is non-empty: $|y| > 0$.
  - The loop happens early: $|xy| \leq p$.

5. Pushdown Automata

Exercise 02

- Consider the language B = { 0^n1^n | n ≥ 0 } over the alphabet Σ = {0, 1}.
- Assume for contradiction that language B is regular with an unknown pumping length p.
- Select the specific string w = 0^p1^p, which belongs to B and satisfies |*w*| ≥ p.
- Determine which characters must make up the substring y under the constraint |*xy*| ≤ p, and show why pumping y creates a contradiction.

5. Pushdown Automata

Exercise 02: Solution

- Assume B is regular with pumping length p, and choose string w = 0^p1^p in B where $|w| \geq p$.
- By the Pumping Lemma, w must split into xyz where $|xy| \leq p$ and $|y| > 0$.
- Because $|xy| \leq p$, the loop segment y must consist strictly of 0s.
- Pumping to i = 2 yields the string xy^2z, which contains more 0s than 1s and thus does not belong to B.
- Accepting a string outside of B violates condition 1 of the Pumping Lemma, proving language B is nonregular.

5. Pushdown Automata

The Power of a Stack

- Finite state machines fail on unbounded counting because they lack a dynamic storage mechanism.
- Adding a Last-In, First-Out (LIFO) stack provides the unbounded auxiliary memory required for tracking.
- A machine can push symbols onto the stack while reading 0s and pop them off while reading 1s.
- Matching succeeds if the stack becomes completely empty at the exact moment the input string ends.
- Incorporating a stack memory structure transforms a Deterministic Finite Automaton into a **Pushdown Automaton**.

5. Pushdown Automata

The Power of a Stack

- Finite state machines fail on unbounded counting because they lack a dynamic storage mechanism.
- Adding a Last-In, First-Out (LIFO) stack provides the unbounded auxiliary memory required for tracking.
- A machine can push symbols onto the stack while reading 0s and pop them off while reading 1s.
- Matching succeeds if the stack becomes completely empty at the exact moment the input string ends.
- Incorporating a stack memory structure transforms a Deterministic Finite Automaton into a **Pushdown Automaton**.

5. Pushdown Automata

How a Stack Solves 0^n1^n

- Read incoming 0 symbols from the input stream and push one marker onto the stack for each occurrence.
- Switch from the pushing phase to the popping phase as soon as the first 1 symbol is encountered.
- Read incoming 1 symbols from the input stream and pop one marker off the stack for each occurrence.
- Reject the input string immediately if a 1 arrives when the stack is already empty.
- **Accept** the input string **if the stack becomes empty** at the precise end of the input string.

5. Pushdown Automata

Formal Definition of a PDA

A Pushdown Automaton (PDA) is defined mathematically as a 6-tuple:

$$(Q, \Sigma, \Gamma, \delta, q_0, F)$$

Components:

- Q: A finite set of states
- Σ: A finite set representing the input alphabet
- Γ: A finite set representing the stack alphabet

δ: The transition function, defined as:

$$\delta : Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \to \mathscr{P}(Q \times \Gamma_\varepsilon)$$

- $q_0 \in Q$: The start state
- $F \subseteq Q$: The set of accept (or final) states

Note: $\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$ and $\Gamma_\varepsilon = \Gamma \cup \{\varepsilon\}$. The power set $\mathscr{P}(Q \times \Gamma_\varepsilon)$ allows for nondeterministic transitions.

5. Pushdown Automata

Formal Definition of a PDA

- The new symbol Γ (Gamma) represents the finite stack alphabet, dictating exactly which symbols can be pushed or popped.
- The transition function δ evaluates your current state, the next input symbol, and the top stack symbol to determine the next move.
- The transition function outputs a set of possible next steps because general PDAs are fundamentally **nondeterministic**.

5. Pushdown Automata

PDA Transition Mechanics and Notation

- Taking a transition requires matching the current state, reading a specific input symbol, and successfully popping a specific top stack symbol.
- **Two-Part Actions**: The result of executing a transition specifies the new state to enter and the new symbol to push onto the stack.
- Transitions are typically written on state diagram arrows using the shorthand format

  "*input, pop → push*".

- A machine can use ε to intentionally skip reading an input symbol, skip popping a stack symbol, or skip pushing a stack symbol.
- Machines traditionally begin by pushing a special bottom-of-stack marker (like $) to easily detect when the stack is empty later.

5. Pushdown Automata

Exercise 03

Design a Pushdown Automaton (PDA) M that recognizes the language $L = \{ 0^n 1^n \mid n \geq 0 \}$. Show its state transition diagram and its formal 6-tuple definition.

Notation Convention: Each transition arrow is labeled as $a, b \rightarrow c$, where:

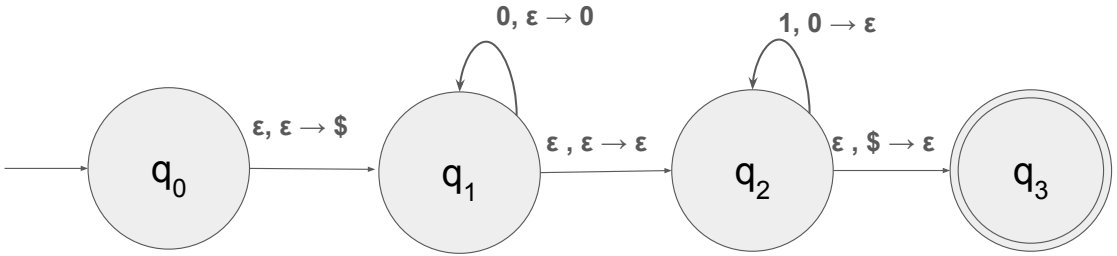
- $\boldsymbol{a} \in \Sigma_\varepsilon$: The input symbol read from the input tape.
- $\boldsymbol{b} \in \Gamma_\varepsilon$: The symbol popped from the top of the stack.
- $\boldsymbol{c} \in \Gamma_\varepsilon$: The symbol pushed onto the top of the stack.

5. Pushdown Automata

Exercise 03: Solution

The PDA M is formally defined as $M = (Q, \Sigma, \Gamma, \delta, q1, F)$, where:

- $Q = \{ q_0, q_1, q_2, q_3 \}$
- $\Sigma = \{ 0, 1 \}$
- $\Gamma = \{ 0, \$ \}$
- $q_0 \in Q$ is the start state
- $F = \{ q_3 \}$



5. Pushdown Automata

Exercise 03: Solution

Current State	Input (a)	Pop (b)	Next State	Push (c)	Formal Notation δ(q,a,b)
q_0	ε	ε	q_1	$	$\delta(q_0, \varepsilon, \varepsilon) = \{ (q_1, \$) \}$
q_1	0	ε	q_1	0	$\delta(q_1, 0, \varepsilon) = \{ (q_1, 0) \}$
q_1	1	0	q_2	ε	$\delta(q_1, 1, 0) = \{ (q_2, \varepsilon) \}$
q_1	ε	$	q_3	ε	$\delta(q_1, \varepsilon, \$) = \{ (q_3, \varepsilon) \}$
q_2	1	0	q_2	ε	$\delta(q_2, 1, 0) = \{ (q_2, \varepsilon) \}$
q_2	ε	$	q_3	ε	$\delta(q_2, \varepsilon, \$) = \{ (q_3, \varepsilon) \}$

5. Pushdown Automata

Exercise 04

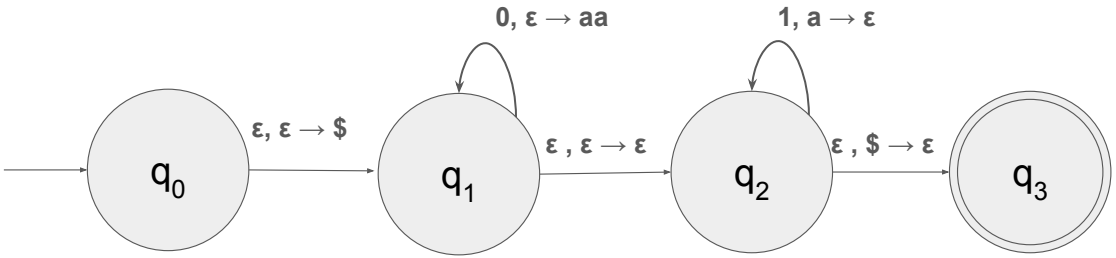
Design a Pushdown Automaton (PDA) M that accepts the language $L_1 = \{ 0^n 1^{2n} \mid n \geq 0 \}$. Provide both the state transition diagram and the formal 6-tuple definition.

5. Pushdown Automata

Exercise 04: Solution

The PDA M is formally defined as $M = (Q, \Sigma, \Gamma, \delta, q_1, F)$, where:

- $Q = \{ q_0, q_1, q_2, q_3 \}$
- $\Sigma = \{ 0, 1 \}$
- $\Gamma = \{ a, \$ \}$
- $q_0 \in Q$ is the start state
- $F = \{ q_3 \}$



5. Pushdown Automata

Exercise 05

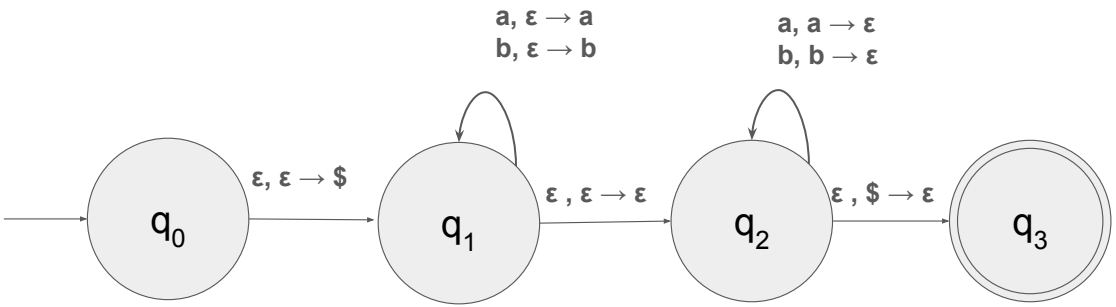
Design a Pushdown Automaton (PDA) that accepts the language $L_2 = \{ ww^R \mid w \in \{a, b\}^* \}$, where w^R is the reverse of string w. Provide both the state transition diagram and the formal 6-tuple definition.

5. Pushdown Automata

Exercise 05: Solution

The PDA M is formally defined as $M = (Q, \Sigma, \Gamma, \delta, q_1, F)$, where:

- $Q = \{ q_0, q_1, q_2, q_3 \}$
- $\Sigma = \{ a, b \}$
- $\Gamma = \{ a, b, \$ \}$
- $q_0 \in Q$ is the start state
- $F = \{ q_3 \}$



5. Pushdown Automata

Exercise 06

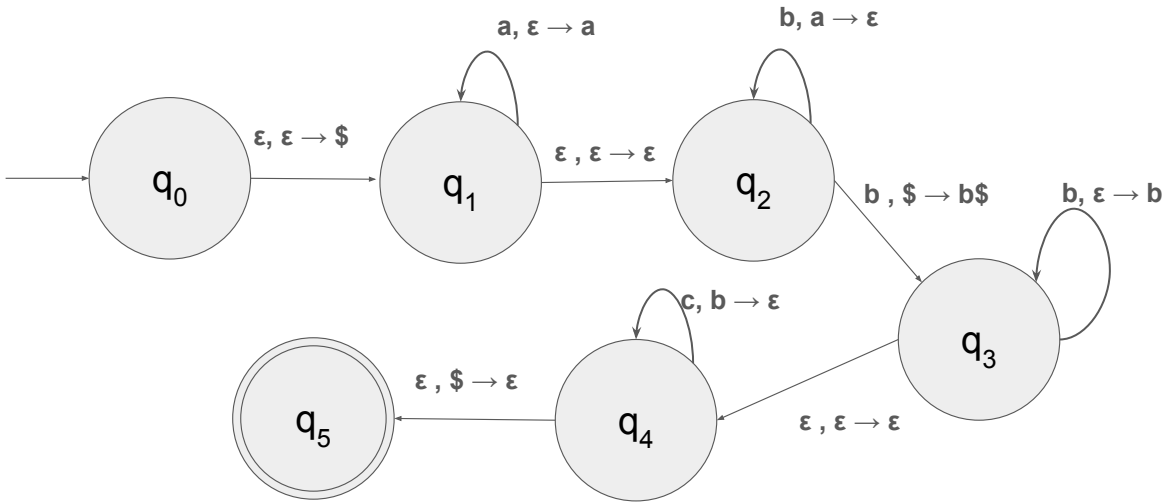
Design a Pushdown Automaton for L1 = { $a^n b^{(n+m)} c^m$ | n, m >= 0 }. Provide both the state transition diagram and the formal 6-tuple definition.

5. Pushdown Automata

Exercise 06: Solution

The PDA M is formally defined as $M = (Q, \Sigma, \Gamma, \delta, q_1, F)$, where:

- $Q = \{ q_0, q_1, q_2, q_3, q_4, q_5 \}$
- $\Sigma = \{ a, b, c \}$
- $\Gamma = \{ a, b, \$ \}$
- $q_0 \in Q$ is the start state
- $F = \{ q_5 \}$



5. Pushdown Automata

Exercise 07

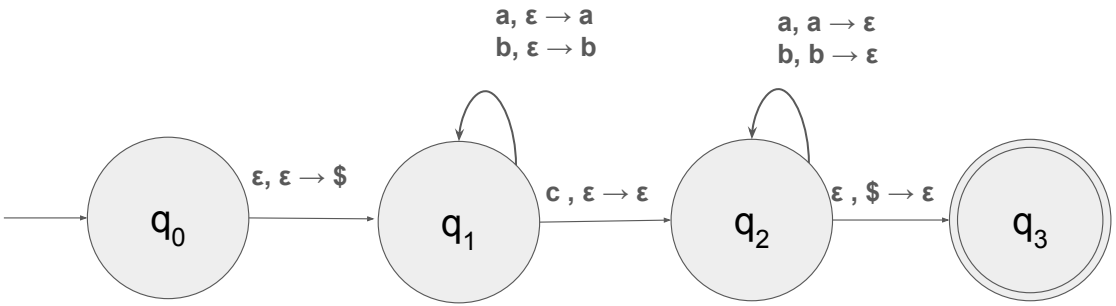
Design a Pushdown Automaton for L2 = { w c w^R | w in {a, b}* }. Use the explicit center symbol 'c' to trigger the state transition without nondeterministic guessing. Provide both the state transition diagram and the formal 6-tuple definition.

5. Pushdown Automata

Exercise 07: Solution

The PDA M is formally defined as $M = (Q, \Sigma, \Gamma, \delta, q_1, F)$, where:

- $Q = \{ q_0, q_1, q_2, q_3 \}$
- $\Sigma = \{ a, b, c \}$
- $\Gamma = \{ a, b, \$ \}$
- $q_0 \in Q$ is the start state
- $F = \{ q_3 \}$



5. Pushdown Automata

Exercise 08

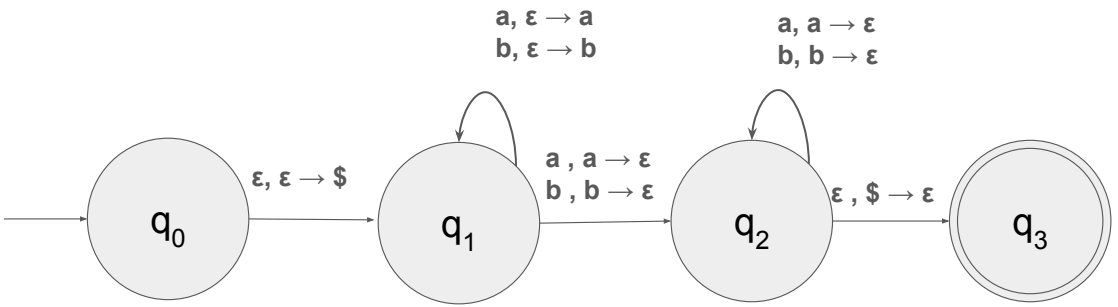
Design a Pushdown Automaton for L = { wxw^R | w in {a, b}*, x in {a, b} }. Provide both the state transition diagram and the formal 6-tuple definition.

5. Pushdown Automata

Exercise 08: Solution

The PDA M is formally defined as $M = (Q, \Sigma, \Gamma, \delta, q_1, F)$, where:

- $Q = \{ q_0, q_1, q_2, q_3 \}$
- $\Sigma = \{ a, b \}$
- $\Gamma = \{ a, b, \$ \}$
- $q_0 \in Q$ is the start state
- $F = \{ q_3 \}$



5. Pushdown Automata

Exercise 09

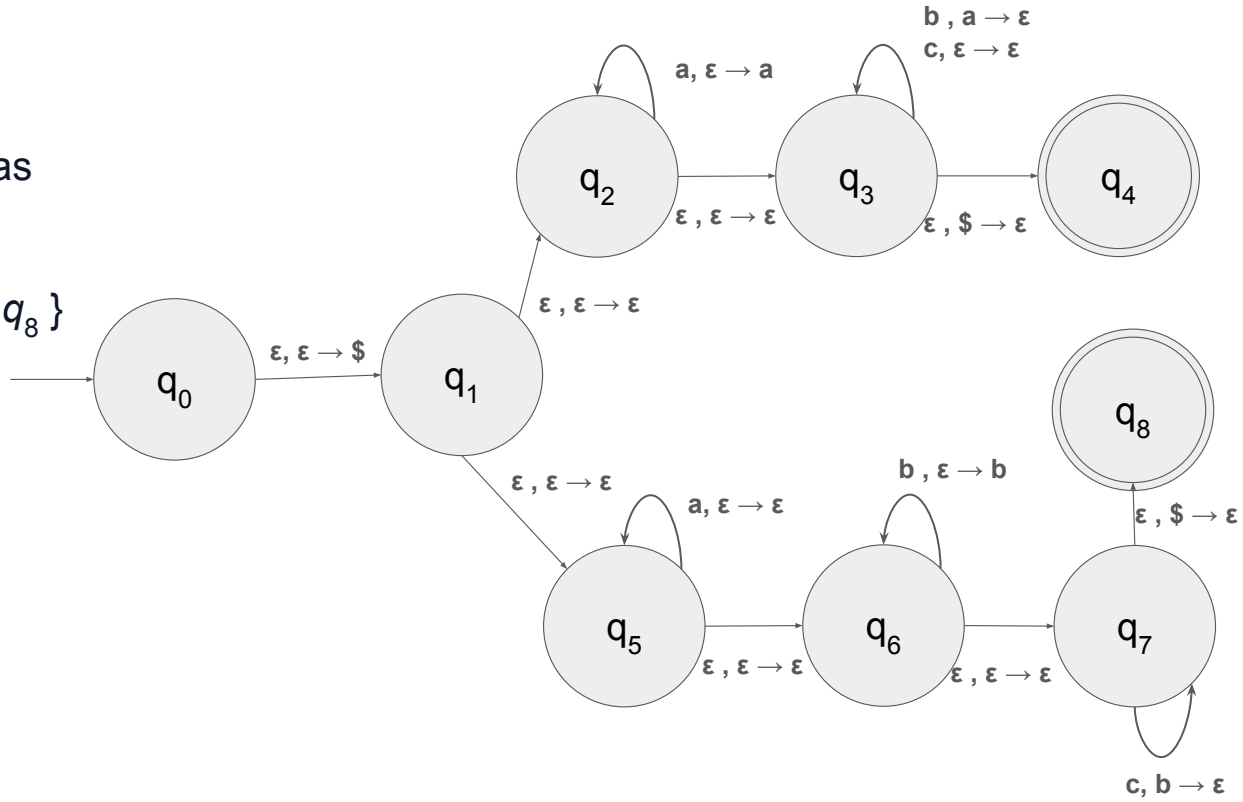
Design a Pushdown Automaton for L3 = { $a^i b^j c^k$ | i = j or j = k }. Provide both the state transition diagram and the formal 6-tuple definition.

5. Pushdown Automata

Exercise 09: Solution

The PDA M is formally defined as $M = (Q, \Sigma, \Gamma, \delta, q_1, F)$, where:

- $Q = \{ q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7, q_8 \}$
- $\Sigma = \{ a, b, c \}$
- $\Gamma = \{ a, b, \$ \}$
- $q_0 \in Q$ is the start state
- $F = \{ q_4, q_8 \}$



5. Pushdown Automata

Exercise 10

Design a Pushdown Automaton for

$$L = \{ w \text{ in } \{0, 1\}^* \mid \text{number of 0s equals number of 1s in any order} \}$$

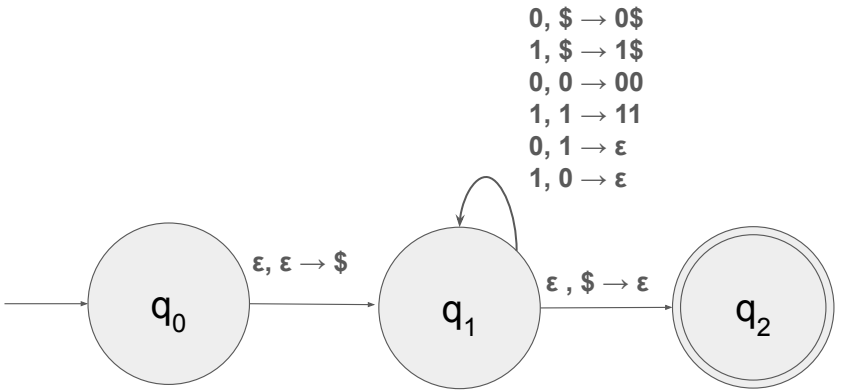
Use the stack to store the current surplus of whichever symbol has appeared more frequently so far.

5. Pushdown Automata

Exercise 10: Solution

The PDA M is formally defined as $M = (Q, \Sigma, \Gamma, \delta, q_1, F)$, where:

- $Q = \{ q_0, q_1, q_2 \}$
- $\Sigma = \{ 0, 1 \}$
- $\Gamma = \{ 0, 1, \$ \}$
- $q_0 \in Q$ is the start state
- $F = \{ q_2 \}$



Questions?

thisaraw@boffin.lk